

DOVE: Double Oil and Vinegar Scheme

V 1.0

Yi Jin*

Yuansheng Pan

Boru Gong

*857078147@qq.com

Contents

1	Introduction	3
2	Preliminaries	3
2.1	Notations	3
2.2	Digital signature scheme	4
2.3	Brief on UOV scheme	4
3	Specification Description	6
3.1	Building block of DOVE	6
3.2	pkc-skc version of DOVE	8
3.3	Classic version of DOVE	9
3.4	Parameter set of DOVE	10
4	Design Rationale	11
4.1	UOV map	11
4.2	DOVE map	11
4.2.1	The Double Vinegar-by-Vinegar Framework	11
4.2.2	Design Intent of the Two Variants: Classic vs. pkc-skc	12
4.2.3	Efficiency Evolution: Outperforming UOV and Rescuing MAYO	12
4.3	Underlying Mathematical Hard Problems	13
4.3.1	The DOVE Multivariate Quadratic (DOVE-MQ) Problem	13
4.3.2	The DOVE Trapdoor Recovery (DOVE-UOV) Problem	14
4.4	Security Proof	14
5	Implementations and Performance	16
5.1	Reference implementation	16
5.2	Performance-optimized implementation	17
6	Concrete Security Analysis	18
6.1	Classical Security Analysis	18
6.1.1	Direct attacks	18
6.1.1.1	Claw finding and collision attacks	18
6.1.1.2	Solve MQ problem	19
6.1.2	Key recovery attacks	20
6.1.2.1	Kipnis-Shamir attack	21
6.1.2.2	Reconciliation attack	21
6.1.2.3	Intersection attack	22
6.1.2.4	Rectangular MinRank attack	22
6.1.3	Wedge, symmetric-algebra, and truncated-ring attacks	23
6.2	Quantum Security Analysis	24
7	Summary and Future Work	25
	References	26

1 Introduction

This document introduces the Double Oil and Vinegar Scheme (DOVE), a digital signature scheme derived from the Unbalanced Oil and Vinegar (UOV) framework. DOVE follows the hash-and-sign paradigm and belongs to the family of Multivariate Public Key Cryptosystems (MPKC), relying on a trapdoored multivariate quadratic map for its security.

The UOV signature scheme, first proposed by Kipnis and Patarin in 1999 [1], is a well-established multivariate cryptosystem. It is highly regarded in the post-quantum cryptography landscape for offering extremely short signature sizes and exceptionally efficient verification processes, as verifying a signature only requires the evaluation of a public quadratic polynomial system.

However, despite these advantages, traditional UOV schemes suffer from a prominent performance bottleneck: the massive public key size and its associated computational and memory overhead. In standard UOV implementations, the public key consists of m independent, large symmetric matrices. Reconstructing these matrices via an Extendable Output Function (XOF) and evaluating them during signing and verification demands m independent and computationally heavy matrix-vector multiplications. More importantly, storing these m massive vinegar $\times$ vinegar matrices during the online execution phase dominates the active memory consumption, which significantly hinders practical performance on memory-constrained devices.

To overcome these limitations and significantly accelerate both signature generation and verification, DOVE introduces a novel algebraic structure into the UOV map via structured matrix factorization. Instead of generating and evaluating m separate vinegar $\times$ vinegar matrices, DOVE parameterizes the system as $m \approx l^2$ and expands only $l = \lceil \sqrt{m} \rceil$ public upper-triangular base matrices. The original independent matrices are then implicitly constructed as the product of two base matrices.

This "Double Vinegar $\times$ Vinegar" design fundamentally shifts the computational paradigm of the scheme, yielding three major benefits:

- **Drastically Reduced Expansion Overhead:** It cuts down the XOF expansion cost, as expanding $\lceil \sqrt{m} \rceil$ base matrices is significantly faster than expanding m full matrices.
- **Pipelined, High-Speed Evaluation:** It transforms the heavy $\mathcal{O}(m \cdot v^2)$ matrix multiplications required in standard UOV into a highly efficient pipelined process. By precomputing a small set of intermediate vectors, the core evaluation of all m equations collapses into lightning-fast vector dot products.
- **Minimal Memory Footprint:** Because the massive vinegar $\times$ vinegar blocks are never explicitly fully reconstructed or stored in memory simultaneously, the peak RAM consumption during signing and verification is drastically lowered.

Ultimately, DOVE successfully preserves the security trapdoor and short signatures of the standard UOV framework while offering a massive leap in computational and memory efficiency for both the signer and the verifier.

2 Preliminaries

2.1 Notations

Let κ denote the security parameter. For a positive integer n , we write $[n] = \{1, \dots, n\}$. A function $\text{negl}(\kappa)$ is said to be *negligible* if it vanishes faster than the inverse of any polynomial in κ .

Let $\mathbb{F}_q$ be a finite field of order q , where q is a prime power. Vectors are denoted by bold lowercase letters, e.g., $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{F}_q^n$. For a vector $\mathbf{x}$, we write $\mathbf{x} \leftarrow \mathcal{D}$ to indicate that $\mathbf{x}$ is sampled according to distribution $\mathcal{D}$; when $\mathcal{D}$ is a set, it means uniform random sampling.

A multivariate quadratic map $\mathcal{F} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ is a tuple of m quadratic polynomials $(f^{(1)}, \dots, f^{(m)})$, where each $f^{(\ell)}(\mathbf{x}) = \sum_{i \leq j} \alpha_{i,j}^{(\ell)} x_i x_j + \sum_i \beta_i^{(\ell)} x_i + \gamma^{(\ell)}$. The polar form of a multivariate quadratic polynomial $f^{(\ell)}$ is the bilinear map $q^{(\ell)}(\mathbf{x}, \mathbf{y}) = f^{(\ell)}(\mathbf{x} + \mathbf{y}) - f^{(\ell)}(\mathbf{x}) - f^{(\ell)}(\mathbf{y}) + f^{(\ell)}(\mathbf{0})$. For the quadratic map $\mathcal{F}$, its polar form is defined componentwise as $\mathcal{Q}(\mathbf{x}, \mathbf{y}) = (q^{(1)}(\mathbf{x}, \mathbf{y}), \dots, q^{(m)}(\mathbf{x}, \mathbf{y}))$.

2.2 Digital signature scheme

Generally speaking, a digital signature scheme $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ consists of three probabilistic polynomial-time algorithms where **KeyGen** is the key generation algorithm, **Sign** is the signing algorithm and **Verify** is the verification algorithm.

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$. pk is a public key and sk is the associated secret key.
- $\sigma \leftarrow \text{Sign}(\text{sk}, \text{msg})$. σ is a signature of the message $\text{msg} \in \{0, 1\}^*$.
- $b := \text{Verify}(\text{pk}, \text{msg}, \sigma)$. It outputs $b \in \{\text{accept}, \text{reject}\}$, indicating whether it accepts the signature σ as a valid signature on msg for the public key pk (i.e., $b = \text{accept}$) or not (i.e., $b = \text{reject}$).

In addition, a digital signature Π may be endowed with another probabilistic polynomial-time algorithm **Setup** running as $\text{params} \leftarrow \text{Setup}(1^\kappa)$.

We say a digital signature scheme $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is *correct*, if for any κ , and any $\text{msg} \in \{0, 1\}^*$, it holds

$$\Pr[\text{Verify}(\text{pk}, \text{msg}, \text{Sign}(\text{sk}, \text{msg})) = \text{accept}] \geq 1 - \text{negl}(\kappa),$$

where the probability is taken over the randomness of the key generation and signing algorithms.

Moreover, the *standard* security definition for a digital signature scheme is existential unforgeability under chosen-message attack (EU-CMA). Given a digital signature scheme $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ and a PPT adversary $\mathcal{A}$ that plays the following game:

1. Challenger runs $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$ and sends pk to $\mathcal{A}$;
2. for $i = 1, \dots, N_s$: $\mathcal{A}$ makes a signing query on msg_i to Challenger and receives $\sigma_i \leftarrow \text{Sign}(\text{sk}, \text{msg}_i)$;
3. $\mathcal{A}$ outputs (msg, σ) .

We say $\mathcal{A}$ *wins* the game if $\text{Verify}(\text{pk}, \text{msg}, \sigma) = \text{accept}$ and $\text{msg} \notin \{\text{msg}_i\}_{i \leq N_s}$, and define the EU-CMA advantage $\text{Adv}_{\text{params}, N_s}^{\text{EU-CMA}}(\mathcal{A})$ of $\mathcal{A}$ against the scheme as the probability that $\mathcal{A}$ wins the game. A scheme is said to have EU-CMA security if for all PPT adversaries, their EU-CMA advantage against the scheme is negligible.

2.3 Brief on UOV scheme

The UOV digital signature scheme was first proposed in 1999 [1]. It leverages the hash-and-sign paradigm underpinned by a trapdoored multivariate quadratic map. Its origin can be traced back to Patarin's groundbreaking linearization equations attack [2] in 1995 against the Matsumoto-Imai cryptosystem [3]. Following this innovative stride, Patarin used the attack's underlying principle to design the Oil and Vinegar signature scheme (OV)

in 1997. However, an invariant subspace attack [4] cracked the *balanced* version of this scheme in 1998. Subsequently, Kipnis, Patarin, and Goubin introduced the Unbalanced Oil and Vinegar (UOV) scheme. Please refer to [5] for more information on UOV. Over two decades, UOV has withstood rigorous cryptanalytic scrutiny, which validates its robust security and consistent reliability.

As a classic multivariate-based scheme, UOV has attracted more and more attention in the past decades. Compared with lattice-based signature schemes, UOV and its variants have good performance, except for the public key size. As such, researchers have been working on proposing variants of UOV to improve efficiency further; for instance, MAYO [6], QR-UOV [7], and SNOVA [8] have been successfully chosen in the second round of the NIST Additional Digital Signature competition.

UOV and its variants follow the same paradigm: hash-and-sign within a structured system of multivariate polynomials. A UOV or its variant signature scheme Π consists of three polynomial-time algorithms (**KeyGen**, **Sign**, **Verify**) and can be specified as follows. Let $\text{Hash} : \{0, 1\}^* \rightarrow \mathbb{F}_q^m$ be a cryptographic hash function.

KeyGen(params = (n, m, q)):

- 1: Choose a central map $\mathcal{F} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ of Oil-Vinegar type (or as required by the variant) uniformly at random.
- 2: Choose an invertible linear map $\mathcal{T} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ uniformly at random.
- 3: $\mathcal{P} := \mathcal{F} \circ \mathcal{T}$.
- 4: **return** ($\text{pk} = \mathcal{P}, \text{sk} = (\mathcal{F}, \mathcal{T})$).

Sign(params, $\text{pk} = \mathcal{P}, \text{sk} = (\mathcal{F}, \mathcal{T}), \text{msg} \in \{0, 1\}^*$):

- 1: $\mathbf{z} := (z_1, \dots, z_m) \leftarrow \text{Hash}(\text{msg})$
- 2: $\mathbf{w} \leftarrow \mathcal{F}^{-1}(\mathbf{z})$
- 3: **return** $\sigma := \mathcal{T}^{-1}(\mathbf{w})$ $\triangleright \mathbf{w} \in \mathbb{F}_q^n$

Verify(params, $\text{pk} = \mathcal{P}, \text{msg} \in \{0, 1\}^*, \sigma$):

- 1: $\mathbf{z} := (z_1, \dots, z_m) \leftarrow \text{Hash}(\text{msg})$
- 2: **return** ($\mathbf{z} == \mathcal{P}(\sigma)$)

Figure 1: A brief description of UOV and its variants.

3 Specification Description

3.1 Building block of DOVE

To minimize storage and transmission overhead, we propose a compressed variant denoted as `DOVE_pkc_sk` (Public Key Compressed, Secret Key Compressed). Following the design paradigm popularized by modern multivariate submissions to the NIST PQC standardization process (such as Rainbow [9] and the UOV submission [10]), both the public and secret keys are expanded from short cryptographically secure seeds via an Extendable Output Function (XOF). The mathematical rationale of the key generation procedure relies on first sampling the secret trapdoor and the public randomized linear blocks, and subsequently solving for the remaining public quadratic terms to satisfy the UOV property [1]. The formal algebraic derivation is described as follows:

1. **Secret Trapdoor Generation:** A master private seed $\text{seed}_{sk} \in \{0, 1\}^{\text{sk_seed_bytes}}$ is chosen uniformly at random. From this seed, the secret basis transformation matrix $\mathbf{T} \in \mathbb{F}_q^{v \times o}$ and the public seed seed_{pk} are pseudo-randomly expanded:

$$(\text{seed}_{pk}, \mathbf{T}) := \text{SM}(\text{seed}_{sk}).$$

2. **Base and Cross-Term Map Expansion:** Using the public seed seed_{pk} , the simulator deterministically expands the $l = \sqrt{m}$ public upper-triangular vinegar-by-vinegar base matrices $\mathbf{A}_k \in \mathbb{F}_q^{v \times v}$ and the m cross-term matrices $\mathbf{P}_{i,2} \in \mathbb{F}_q^{v \times o}$. To maintain the secret trapdoor relations, the signer internally computes the intermediate matrices $\mathbf{B}_k = \mathbf{T}^t \mathbf{A}_k$ and $\mathbf{C}_k = \mathbf{A}_k \mathbf{T}$.
3. **Oil-by-Oil Block Construction:** In the standard UOV framework, the secret trapdoor is characterized by an isomorphic linear transformation matrix $\mathbf{M}_T \in \mathbb{F}_q^{n \times n}$, defined in a block-structured form via the secret matrix $\mathbf{T} \in \mathbb{F}_q^{v \times o}$ as:

$$\mathbf{M}_T = \begin{pmatrix} \mathbf{I}_v & \mathbf{T} \\ \mathbf{0} & \mathbf{I}_o \end{pmatrix},$$

where $\mathbf{I}_v$ and $\mathbf{I}_o$ denote the identity matrices of order v and o , respectively. Let $\mathbf{F}_i \in \mathbb{F}_q^{n \times n}$ be the central map quadratic matrix and $\mathbf{P}_i \in \mathbb{F}_q^{n \times n}$ be the public key quadratic matrix for the i -th equation. They are structurally partitioned into vinegar and oil blocks as follows:

$$\mathbf{F}_i = \begin{pmatrix} \mathbf{F}_{i,1} & \mathbf{F}_{i,2} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}, \quad \mathbf{P}_i = \begin{pmatrix} \mathbf{P}_{i,1} & \mathbf{P}_{i,2} \\ \mathbf{0} & \mathbf{P}_{i,3} \end{pmatrix},$$

where the oil-by-oil block of the central map $\mathbf{F}_{i,3}$ is explicitly set to $\mathbf{0}$ to embed the UOV trapdoor, and we choose to represent these maps as upper-triangular or equivalent block-structured matrices. The relationship between the public map and the central map is governed by the equivalence relation $\mathbf{F}_i = \mathbf{M}_T^\top \mathbf{P}_i \mathbf{M}_T$. To derive the precise algebraic constraint on the public blocks, we expand this block matrix multiplication explicitly:

$$\begin{aligned} \mathbf{F}_i &= \mathbf{M}_T^\top \mathbf{P}_i \mathbf{M}_T \\ &= \begin{pmatrix} \mathbf{I}_v & \mathbf{0} \\ \mathbf{T}^t & \mathbf{I}_o \end{pmatrix} \begin{pmatrix} \mathbf{P}_{i,1} & \mathbf{P}_{i,2} \\ \mathbf{0} & \mathbf{P}_{i,3} \end{pmatrix} \begin{pmatrix} \mathbf{I}_v & \mathbf{T} \\ \mathbf{0} & \mathbf{I}_o \end{pmatrix} \\ &= \begin{pmatrix} \mathbf{P}_{i,1} & \mathbf{P}_{i,2} \\ \mathbf{T}^\top \mathbf{P}_{i,1} & \mathbf{T}^\top \mathbf{P}_{i,2} + \mathbf{P}_{i,3} \end{pmatrix} \begin{pmatrix} \mathbf{I}_v & \mathbf{T} \\ \mathbf{0} & \mathbf{I}_o \end{pmatrix} \\ &= \begin{pmatrix} \mathbf{P}_{i,1} & \mathbf{P}_{i,1} \mathbf{T} + \mathbf{P}_{i,2} \\ \mathbf{T}^\top \mathbf{P}_{i,1} & \mathbf{T}^\top \mathbf{P}_{i,1} \mathbf{T} + \mathbf{T}^\top \mathbf{P}_{i,2} + \mathbf{P}_{i,3} \end{pmatrix}. \end{aligned}$$

Hence, we have

$$\mathbf{0} = \text{Upper}(\mathbf{T}^\top \mathbf{P}_{i,1} \mathbf{T} + \mathbf{T}^\top \mathbf{P}_{i,2} + \mathbf{P}_{i,3}).$$

Substituting our factorized structure $\mathbf{P}_{i,1} \sim \mathbf{A}_{k_1} \mathbf{A}_{k_2}$ and isolating $\mathbf{P}_{i,3}$, the components are uniquely determined by:

$$\mathbf{P}_{i,3} = \text{Upper}(-\mathbf{B}_{k_1} \mathbf{C}_{k_2} - \mathbf{T}^t \mathbf{P}_{i,2}),$$

where $k_1 = \lfloor i/l \rfloor$ and $k_2 = i \pmod l$.

Consequently, the public key is compressed to $\text{pk} = (\text{seed}_{pk}, \{\mathbf{P}_{i,3}\}_{i \in [m]})$, where the massive $\mathbf{A}_k$ and $\mathbf{P}_{i,2}$ matrices are omitted and regenerated on-the-fly during signature verification. The secret key is fully encapsulated by $\text{sk} = \text{seed}_{sk}$. The complete algorithmic execution for key generation, signing, and verification is formalized in Algorithm 4.

```

Upper( $\mathbf{A} \in \mathbb{F}_q^{n \times n}$ ):
1: for  $i$  from 0 to  $n - 1$  do
2:   for  $j$  from  $i + 1$  to  $n - 1$  do
3:      $\mathbf{A}[i, j] += \mathbf{A}[j, i]$ 
4:      $\mathbf{A}[j, i] = 0$ 
5: return  $\mathbf{A}$ .

```

Figure 2: Upper algorithm.

In contrast to the seed-compressed framework, we also define the `DOVE_classic` variant, which is strictly tailored for hardware or cloud environments prioritizing maximum, uncompromised computational speed.

The core philosophy of `DOVE_classic` relies on the observation that all structural matrices inherited from the UOV framework can be fully precomputed. In this variant, the potentially heavy cryptographic XOF expansion function $\text{SM}(\cdot)$ used to generate the structural components—specifically $\{\mathbf{A}_k\}$, $\{\mathbf{B}_k\}$, $\{\mathbf{C}_k\}$, and $\{\mathbf{P}_{i,2}\}$ —is completely banished from the online execution flow. Instead, these voluminous components are expanded **exactly once** during the offline `KeyGen` phase and stored explicitly within the expanded key structures.

3.2 pkc-skc version of DOVE

```

KeyGen():
1: seedsk ← {0, 1}sk_seed_bytes
2: seedpk, T := SM3(seedsk, pk_seed_bytes + ⌈v * o *  $\frac{r}{8}$ ⌉) ▷ T ∈  $\mathbb{F}_q^{v \times o}$ 
3: for k from 0 to l - 1 do
4:   Ak := SM3(seedpk || k, ⌈ $\frac{v*(v+1)}{2} \frac{r}{8}$ ⌉). ▷ upper-triangular Ak ∈  $\mathbb{F}_q^{v \times v}$ 
5:   Bk = TtAk, Ck = AkT ▷ Bk ∈  $\mathbb{F}_q^{o \times v}$ , Ck ∈  $\mathbb{F}_q^{v \times o}$ 
6: for i from 0 to m - 1 do
7:   Pi,2 := SM3(seedpk || l + i, ⌈v * o *  $\frac{r}{8}$ ⌉) ▷ Pi,2 ∈  $\mathbb{F}_q^{v \times o}$ 
8:   k1 = ⌊i/l⌋, k2 = i - k1 * l
9:   Pi,3 = Upper(-Bk1Ck2 - TtPi,2)
10: return (pk = (seedpk, {Pi,3}i∈[m]), sk = seedsk).

Sign(sk = seedsk, msg ∈ {0, 1}*):
1: seedpk, T := SM3(seedsk, pk_seed_bytes + ⌈v * o *  $\frac{r}{8}$ ⌉)
2: t := (t1, ..., tm) ← SM3(msg || seedpk, ⌈m *  $\frac{r}{8}$ ⌉) ▷ T ∈  $\mathbb{F}_q^{v \times o}$ 
3: salt := SM3(seedsk || t, salt_bytes)
4: for ctr from 0 to 255 do
5:   v := (v1, ..., vv) ← SM3(t || seedsk || ctr, ⌈v *  $\frac{r}{8}$ ⌉) ▷ v ∈  $\mathbb{F}_q^v$ 
6:   for k from 0 to l - 1 do
7:     Ak := SM3(seedpk || k, ⌈ $\frac{v*(v+1)}{2} \frac{r}{8}$ ⌉). ▷ upper-triangular Ak ∈  $\mathbb{F}_q^{v \times v}$ 
8:     xk = vtAk, yk = Akv
9:     Bk = TtAk, Ck = AkT
10:    for i from 0 to m - 1 do
11:      Pi,2 := SM3(seedpk || l + i, ⌈v * o *  $\frac{r}{8}$ ⌉)
12:      k1 = ⌊i/l⌋, k2 = i - k1 * l
13:      w[i] := xk1yk2
14:      L[i, :] = vtPi,2 + xk1Ck2 + yk2tBk1
15:    if L invertible then
16:      z = L-1(t - w)
17:      s = (v + Tz) || z
18:      return σ := (s, salt) ▷ s ∈  $\mathbb{F}_q^n$ 
19: return 0

Verify(pk = (seedpk, {Pi,3}i∈[m]), msg ∈ {0, 1}*, σ = (s, salt)):
1: t := (t1, ..., tm) ← SM3(msg || seedpk, ⌈m *  $\frac{r}{8}$ ⌉)
2: v = s[0 : v - 1], z = s[v : n - 1]
3: for k from 0 to l - 1 do
4:   Ak := SM3(seedpk || k, ⌈ $\frac{v*(v+1)}{2} \frac{r}{8}$ ⌉). ▷ upper-triangular Ak ∈  $\mathbb{F}_q^{v \times v}$ 
5:   xk = vtAk
6:   yk = Akv
7: for i from 0 to m - 1 do
8:   Pi,2 := SM3(seedpk || l + i, ⌈v * o *  $\frac{r}{8}$ ⌉)
9:   k1 = ⌊i/l⌋, k2 = i - k1 * l
10:  w[i] := xk1yk2 + vtPi,2z + ztPi,3z
11: return (t == w)

```

Figure 3: Key generation, signature generation and verification algorithm of DOVE_pkc_skc

3.3 Classic version of DOVE

```

KeyGen():
1: seedsk ← {0, 1}sk_seed_bytes
2: seedpk, T := SM3(seedsk, pk_seed_bytes + ⌈v * o *  $\frac{r}{8}$ ⌉) ▷ T ∈  $\mathbb{F}_q^{v \times o}$ 
3: for k from 0 to l - 1 do
4:   Ak := SM3(seedpk || k, ⌈ $\frac{v * (v+1)}{2} \frac{r}{8}$ ⌉). ▷ upper-triangular Ak ∈  $\mathbb{F}_q^{v \times v}$ 
5:   Bk = Tt Ak ▷ Bk ∈  $\mathbb{F}_q^{o \times v}$ 
6:   Ck = Ak T ▷ Ck ∈  $\mathbb{F}_q^{v \times o}$ 
7: for i from 0 to m - 1 do
8:   Pi,2 := SM3(seedpk || l + i, ⌈v * o *  $\frac{r}{8}$ ⌉) ▷ Pi,2 ∈  $\mathbb{F}_q^{v \times o}$ 
9:   k1 = ⌊i/l⌋, k2 = i - k1 * l
10:  Pi,3 = Upper(-Bk1 Ck2 - Tt Pi,2)
11: return pk = (seedpk, {Ak}k∈[l], {Pi,2}i∈[m], {Pi,3}i∈[m])
      sk = (seedsk, seedpk, T, {Ak}k∈[l], {Bk}k∈[l], {Ck}k∈[l], {Pi,2}i∈[m])

Sign (sk = (seedsk, T, {Ak}k∈[l], {Bk}k∈[l], {Ck}k∈[l], {Pi,2}i∈[m]), msg ∈ {0, 1}*):
1: t := (t1, ..., tm) ← SM3(msg || seedpk, ⌈m *  $\frac{r}{8}$ ⌉) ▷ T ∈  $\mathbb{F}_q^{v \times o}$ 
2: salt := SM3(seedsk || t, salt_bytes)
3: for ctr from 0 to 255 do
4:   v := (v1, ..., vv) ← SM3(t || seedsk || ctr, ⌈v *  $\frac{r}{8}$ ⌉) ▷ v ∈  $\mathbb{F}_q^v$ 
5:   for k from 0 to l - 1 do
6:     xk = vt Ak
7:     yk = Ak v
8:   for i from 0 to m - 1 do
9:     k1 = ⌊i/l⌋, k2 = i - k1 * l
10:    w[i] := xk1 yk2
11:    L[i, :] = vt Pi,2 + xk1 Ck2 + yk2 Bk1t
12:    if L invertible then
13:      z = L-1(t - w)
14:      s = (v + Tz) || z
15:      return σ := (s, salt) ▷ s ∈  $\mathbb{F}_q^n$ 
16: return 0

Verify (pk = (seedpk, {Ak}k∈[l], {Pi,2}i∈[m], {Pi,3}i∈[m]), msg ∈ {0, 1}*, σ = (s, salt)):
1: t := (t1, ..., tm) ← SM3(msg || seedpk, ⌈m *  $\frac{r}{8}$ ⌉)
2: v := s[0 : v - 1]
3: z := s[v : n - 1]
4: for k from 0 to l - 1 do
5:   xk = vt Ak
6:   yk = Ak v
7: for i from 0 to m - 1 do
8:   k1 = ⌊i/l⌋, k2 = i - k1 * l
9:   w[i] := xk1 yk2 + vt Pi,2 z + zt Pi,3 z
10: return (t == w)

```

Figure 4: Key generation, signature generation and verification algorithm of DOVE_classic

3.4 Parameter set of DOVE

We take field $\mathbb{F}_q = \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1)$. For all numeric values exceeding two bytes in size, they shall be represented in little-endian byte order (least significant byte first) when fed into the hash function.

The public key size is $\text{pk_seed_bytes} + \lceil m * \frac{o(o+1)}{2} * \frac{r}{8} \rceil$ (bytes) and the secret key size is sk_seed_bytes (bytes).

Table 1: Chosen parameters of DOVE

Parameter set	DOVE ₁₂₈	DOVE ₂₅₆	DOVE ₅₁₂
$o = m$	44	96	216
$n = v + o$	112	256	576
$q = 2^r$	256	256	256
l	7	10	15
salt_bytes	24	40	72
sk_seed_bytes	24	40	72
pk_seed_bytes	16	16	16

Table 2: Key and signature sizes for DOVE parameter sets

Parameter set	Public key (B)	Private key (B)	Signature (B)
DOVE_classic_128	191,646	192,990	136
DOVE_classic_256	2,050,352	1,925,976	296
DOVE_classic_512	22,833,052	20,181,508	648
DOVE_pkc_sk_128	43,576	24	136
DOVE_pkc_sk_256	446,992	40	296
DOVE_pkc_sk_512	5,062,192	72	648

4 Design Rationale

Our scheme follows a structure similar to that of UOV, but to mitigate the large public key size of UOV, we introduce a "Double Vinegar $\times$ Vinegar" structure into the UOV map. Below we first recall the definition and properties of the UOV map.

4.1 UOV map

The central object in the Unbalanced Oil and Vinegar (UOV) digital signature scheme is a trapdoored one-way multivariate quadratic map $\mathcal{P} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$, known as the *UOV map*. This map serves as the public key.

The special structure of the UOV map is as follows. There exists an o -dimensional subspace $\mathcal{O} \subseteq \mathbb{F}_q^n$ (called the *oil space*), with $o \geq m$, on which $\mathcal{P}$ vanishes completely: $\mathcal{P}(\mathbf{o}) = \mathbf{0}$ for every $\mathbf{o} \in \mathcal{O}$. The complementary $v = n - o$ dimensions are called the *vinegar space*. The oil space is the trapdoor. To invert $\mathcal{P}$ for any vector $\mathbf{z} \in \mathbb{F}_q^m$, picks a random vector $\mathbf{v} \in \mathbb{F}_q^n$ and solves the linear system obtained by evaluating $\mathcal{P}$ at $\mathbf{o} + \mathbf{v}$, which becomes linear in the oil variables $\mathbf{o}$ precisely because $\mathcal{P}$ vanishes on $\mathcal{O}$. More concretely, using the polar form associated with $\mathcal{P}$, one has

$$\mathcal{P}(\mathbf{o} + \mathbf{v}) = \mathcal{Q}(\mathbf{o}, \mathbf{v}) + \mathcal{P}(\mathbf{v}) + \mathcal{P}(\mathbf{o}) = \mathcal{Q}(\mathbf{o}, \mathbf{v}) + \mathcal{P}(\mathbf{v}),$$

since $\mathcal{P}(\mathbf{o}) = 0$. Here $\mathcal{Q}(\mathbf{o}, \mathbf{v})$ is linear in $\mathbf{o}$. Setting $\mathcal{P}(\mathbf{o} + \mathbf{v}) = \mathbf{z}$ yields a system of m linear equations in o unknowns, which can be solved efficiently as long as $o \geq m$. The signature is then $\sigma = \mathbf{o} + \mathbf{v}$. This trapdoor mechanism is what makes UOV practical.

Thus, the UOV map $\mathcal{P}$ is not an arbitrary quadratic map; it is specifically crafted to hide an o -dimensional vanishing subspace (the oil space). The security of UOV relies on the hardness of recovering this subspace from the description of $\mathcal{P}$ alone — a problem known as the UOV problem. Conversely, the efficiency of signing exploits the same subspace.

4.2 DOVE map

The primary design objective of the DOVE (Double Oil and Vinegar Scheme) signature scheme is to overcome the massive public key size and heavy computational overhead inherent in traditional Unbalanced Oil and Vinegar (UOV) schemes.

In a standard UOV scheme, the public key comprises m multivariate quadratic equations, each represented by a symmetric matrix $\mathbf{P}_i \in \mathbb{F}_q^{n \times n}$ structured as:

$$\mathbf{P}_i = \begin{pmatrix} \mathbf{P}_{i,1} & \mathbf{P}_{i,2} \\ \mathbf{0} & \mathbf{P}_{i,3} \end{pmatrix} \quad (1)$$

where $\mathbf{P}_{i,1} \in \mathbb{F}_q^{v \times v}$ represents the vinegar-by-vinegar quadratic terms, $\mathbf{P}_{i,2} \in \mathbb{F}_q^{v \times o}$ represents the vinegar-by-oil cross terms, and $\mathbf{P}_{i,3} \in \mathbb{F}_q^{o \times o}$ represents the oil-by-oil quadratic terms. Reconstructing and evaluating m independent large $\mathbf{P}_{i,1}$ matrices from a seed via an Extendable Output Function (XOF) constitutes the primary performance bottleneck in both signing and verification.

4.2.1 The Double Vinegar-by-Vinegar Framework

DOVE bypasses this bottleneck by introducing a structured matrix factorization technique. Instead of generating m separate $\mathbf{P}_{i,1}$ matrices, the scheme parameterizes the number of equations as m and expands only $l = \lceil \sqrt{m} \rceil$ base upper-triangular matrices $\mathbf{A}_k \in \mathbb{F}_q^{v \times v}$ (for $k \in \{0, \dots, l-1\}$) from the public seed seed_{pk} .

For the i -th equation, the index i is mapped to a unique pair of base matrices via the index decomposition $k_1 = \lfloor i/l \rfloor$ and $k_2 = i \pmod l$. The scheme implicitly constructs the vinegar-by-vinegar map as the product of these two base matrices:

$$\mathbf{P}_{i,1} \sim \mathbf{A}_{k_1} \mathbf{A}_{k_2} \quad (2)$$

To maintain the standard UOV trapdoor relation under the secret linear map $\mathbf{T} \in \mathbb{F}_q^{v \times o}$, the public oil-by-oil block $\mathbf{P}_{i,3}$ is derived during **KeyGen** by eliminating the secret vinegar-to-vinegar terms:

$$\mathbf{P}_{i,3} = \text{Upper} \left(-(\mathbf{T}^t \mathbf{A}_{k_1})(\mathbf{A}_{k_2} \mathbf{T}) - \mathbf{T}^t \mathbf{P}_{i,2} \right) = \text{Upper} \left(-\mathbf{B}_{k_1} \mathbf{C}_{k_2} - \mathbf{T}^t \mathbf{P}_{i,2} \right) \quad (3)$$

where $\mathbf{B}_{k_1} = \mathbf{T}^t \mathbf{A}_{k_1}$ and $\mathbf{C}_{k_2} = \mathbf{A}_{k_2} \mathbf{T}$ are precomputed intermediate secret matrices. This elegant algebraic trick ensures that the structural modification seamlessly preserves the standard UOV multivariate trapdoor functionality.

4.2.2 Design Intent of the Two Variants: Classic vs. pkc-sk

To accommodate different real-world application scenarios, DOVE is designed with two distinct variants targeting different trade-offs between key storage and execution speed:

- **The Classic Variant (DOVE_classic):** This version is tailored for environments prioritizing maximum computational speed. As observed in the pseudocode of `DOVE_classic`, the potentially heavy XOF function $\text{SM}(\cdot)$ for structural matrices $(\{\mathbf{A}_k\}, \{\mathbf{B}_k\}, \{\mathbf{C}_k\}, \{\mathbf{P}_{i,2}\})$ is completely absent from the online **Sign** and **Verify** phases. Instead, these components are expanded **exactly once** during the **KeyGen** phase and stored explicitly within the keys.

For memory-constrained platforms where storing full keys is prohibited, these matrices can be expanded on-the-fly upon the first invocation of the signing/verification module and cached locally for subsequent runs. By isolating the expansion operations outside the central algorithmic loop, the execution of **Sign** and **Verify** scales down to clean, straightforward matrix arithmetic, making it drastically faster than standard compressed UOV variants.

- **The Compressed Variant (DOVE_pkc_sk):** This version acts as a low-bandwidth paradigm where public and secret components are compactly stored as small seeds (seed_{sk} and seed_{pk}). Although it requires on-the-fly regeneration of matrices inside the signing and verification loops, its matrix decomposition ensures that the expansion cost scales with $l = \lceil \sqrt{m} \rceil$ instead of m . Thus, even in its fully compressed state, it remains computationally lightweight.

4.2.3 Efficiency Evolution: Outperforming UOV and Rescuing MAYO

The structural design choices of DOVE yield profound computational advantages when analyzed alongside traditional UOV and modern compact frameworks like MAYO[6]:

1. **Algorithmic Complexity Reduction over Standard UOV:** In traditional UOV, evaluating the quadratic vinegar component $\mathbf{w}[i] = \mathbf{v}^t \mathbf{P}_{i,1} \mathbf{v}$ for all m equations requires executing m independent vector-matrix-vector multiplications, yielding an immutable complexity of $\mathcal{O}(m \cdot v^2)$.

DOVE maps this problem into a highly efficient pipelined optimization. Before computing the equations, it pre-calculates two sets of vector pools:

$$\mathbf{x}_k = \mathbf{v}^t \mathbf{A}_k \quad \text{and} \quad \mathbf{y}_k = \mathbf{A}_k \mathbf{v} \quad \text{for } k \in \{0, \dots, l-1\} \quad (4)$$

This initial step requires only $2l = 2\sqrt{m}$ vector-matrix products, costing $\mathcal{O}(\sqrt{m} \cdot v^2)$ field operations. Once cached, the vinegar evaluation for the i -th equation is reduced to a single vector dot product $\mathbf{w}[i] = \mathbf{x}_{k_1} \mathbf{y}_{k_2}$, which takes a mere $\mathcal{O}(m \cdot v)$ operations over all m channels. The dominant complexity thereby falls from $\mathcal{O}(m \cdot v^2)$ to $\mathcal{O}(\sqrt{m} \cdot v^2 + m \cdot v)$, liberating massive amounts of processor cycles.

2. **Mitigating the Performance Bottleneck of MAYO:** MAYO is a contemporary state-of-the-art multivariate scheme that achieves exceptionally compact public keys by utilizing a technique known as *whipping*. Instead of choosing standard large UOV dimensions, MAYO selects small (v, o) parameters and expands the map artificially by evaluating the core public maps against multiple linear combinations of different vectors simultaneously.

While whipping solves the public key size dilemma, it inflicts a severe performance penalty. To generate or verify a signature, the algorithms must evaluate the core vinegar maps $\mathbf{P}_{i,1}$ against a vast combinatorial arrangement of multiple vector pairs. This creates an intense computational bottleneck due to the repetitive execution of m matrix-vector multiplications for every whipping path, causing MAYO to be significantly slower than UOV.

DOVE's framework offers a drop-in remedy for this exact bottleneck. When applied to a whipping architecture, instead of computing m matrix multiplications per path, DOVE only needs to calculate $2l = 2\sqrt{m}$ matrix-vector products to construct the $\mathbf{x}_k$ and $\mathbf{y}_k$ vector pools for each unique vector instance. All subsequent evaluations across different whipped cross-combinations dissolve into ultra-cheap vector dot products. Consequently, integrating DOVE's design into a MAYO-like structure perfectly mitigates the efficiency penalties of whipping, paving the way for signature schemes that simultaneously achieve highly compressed public keys and optimal execution speeds.

4.3 Underlying Mathematical Hard Problems

The security of the DOVE signature scheme is asymptotically reduced to the hardness of two distinct mathematical problems: a structural variant of the Multivariate Quadratic (MQ) problem, which guarantees resistance against direct forgery attacks, and a structured trapdoor recovery problem, which ensures resistance against key recovery attacks.

4.3.1 The DOVE Multivariate Quadratic (DOVE-MQ) Problem

The foundational cryptographic hardness of multivariate schemes relies on the NP-hardness of solving a random system of quadratic equations over a finite field. For DOVE, the public equations are not entirely random but possess a specific algebraic structure within their vinegar-by-vinegar maps. We formally define the DOVE-MQ problem as follows. Let n, m, l, v, o be the operational parameters defined by the system such that $l = \lceil \sqrt{m} \rceil$ and $n = v + o$. Let $\mathcal{P} = (P_1, \dots, P_m) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ be a multivariate quadratic map where each polynomial component $P_i(\mathbf{x})$ for an input vector $\mathbf{x} = (\mathbf{v} \parallel \mathbf{z}) \in \mathbb{F}_q^v \times \mathbb{F}_q^o$ is explicitly structured as:

$$P_i(\mathbf{x}) = \mathbf{v}^t \mathbf{A}_{k_1} \mathbf{A}_{k_2} \mathbf{v} + \mathbf{v}^t \mathbf{P}_{i,2} \mathbf{z} + \mathbf{z}^t \mathbf{P}_{i,3} \mathbf{z}$$

where $k_1 = \lfloor i/l \rfloor$, $k_2 = i \pmod{l}$, the base matrices $\mathbf{A}_k \in \mathbb{F}_q^{v \times v}$ are public upper-triangular matrices, $\mathbf{P}_{i,2} \in \mathbb{F}_q^{v \times o}$, and $\mathbf{P}_{i,3} \in \mathbb{F}_q^{o \times o}$.

Definition 1. (DOVE-MQ Problem) Given a public quadratic map $\mathcal{P}$ sampled according to the DOVE structural distribution and a target vector $\mathbf{t} \in \mathbb{F}_q^m$, find a vector $\mathbf{s} \in \mathbb{F}_q^n$ such that $\mathcal{P}(\mathbf{s}) = \mathbf{t}$.

The DOVE-MQ problem asserts that finding a preimage for this structured quadratic map remains as difficult as solving a generic random MQ system via state-of-the-art algebraic solvers (e.g., the F_4/F_5 algorithms or Wiedemann XL), as the base matrix decomposition $\mathbf{A}_{k_1}\mathbf{A}_{k_2}$ does not introduce lower-degree algebraic relations or linear dependencies available to an attacker.

4.3.2 The DOVE Trapdoor Recovery (DOVE-UOV) Problem

While the DOVE-MQ problem ensures that an adversary cannot directly forge a valid signature for a given message, the scheme must also guarantee that the secret trapdoor cannot be leaked by the public key description.

Definition 2. (DOVE-UOV Problem) Given a public key $\text{pk} = (\text{seed}_{\text{pk}}, \{\mathbf{P}_{i,3}\}_{i \in [m]})$ generated via KeyGen , where the public blocks satisfy the relation $\mathbf{P}_{i,3} = \text{Upper}(-\mathbf{B}_{k_1}\mathbf{C}_{k_2} - \mathbf{T}^t\mathbf{P}_{i,2})$, find a matrix $\mathbf{T}' \in \mathbb{F}_q^{v \times o}$ or an equivalent o -dimensional subspace $\mathcal{O}' \subset \mathbb{F}_q^n$ that successfully acts as a valid UOV trapdoor for the map $\mathcal{P}$.

The DOVE-UOV problem generalizes the standard UOV trapdoor recovery task. An attacker attempting to solve this problem must cope not only with the hidden oil space but also with the structural correlation imposed by the fact that the secret intermediate components $\mathbf{B}_{k_1}$ and $\mathbf{C}_{k_2}$ are tightly bounded by the shared public base matrices $\mathbf{A}_k$. This problem underpins the key-recovery security of the scheme against specialized structural attacks, such as the Kipnis-Shamir attack and intersection attacks.

4.4 Security Proof

In this section, we prove that the DOVE signature scheme is Existentially Unforgeable under Chosen Message Attacks (EUF-CMA) in the Random Oracle Model (ROM), tightly reduced to the DOVE-MQ and DOVE-UOV hard problems.

Theorem 1. *Let $\mathcal{H}$ be a hash function modeled as a random oracle. If the DOVE-MQ problem and the DOVE-UOV problem are intractable, then the DOVE signature scheme is EUF-CMA secure.*

Proof. Let $\mathcal{A}$ be a Probabilistic Polynomial-Time (PPT) adversary that makes at most q_h queries to the random oracle $\mathcal{H}$ and at most q_s signature queries. We construct the proof using a sequence of games, starting from the real attack scenario and ending in a simulated scenario where forgery directly implies solving the DOVE-MQ problem.

- **Game 0 (The Real Game):** This is the standard EUF-CMA security game. The challenger generates a valid key pair $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ and provides pk to the adversary $\mathcal{A}$. The challenger answers $\mathcal{A}$'s hash queries $\mathcal{H}(\text{msg})$ and signature queries $\text{Sign}(\text{sk}, \text{msg})$ faithfully according to the real algorithms. By definition, the probability that $\mathcal{A}$ wins Game 0 is $\text{Adv}_{\text{DOVE}}^{\text{EUF-CMA}}(\mathcal{A})$.
- **Game 1 (Trapdoor Indistinguishability):** In this game, we modify the generation of the public key. Instead of deriving the oil-by-oil blocks $\mathbf{P}_{i,3}$ from the secret trapdoor matrices $\mathbf{B}_k, \mathbf{C}_k$ and $\mathbf{T}$ (which hides the oil subspace), the challenger uniformly samples random symmetric matrices $\mathbf{P}_{i,3} \xleftarrow{\$} \mathbb{F}_q^{o \times o}$ to construct a structurally valid but completely trapdoor-less DOVE map $\mathcal{P}^*$.

Since the generation of $\mathbf{P}_{i,3}$ is the only difference between Game 0 and Game 1, if $\mathcal{A}$'s success probability or behavior changes noticeably, it can be used to build an algorithm $\mathcal{B}_1$ that distinguishes a real DOVE public key from a trapdoor-less map. This is precisely the definition of the DOVE-UOV problem. Therefore, the advantage difference is bounded by:

$$|\Pr[S_0] - \Pr[S_1]| \leq \text{Adv}^{\text{DOVE-UOV}}(\mathcal{B}_1)$$

- **Game 2 (Random Oracle Programming and Direct Forgery):** Since the public key in Game 1 no longer contains a valid trapdoor, the challenger can no longer use the real **Sign** algorithm to answer signature queries. Instead, the challenger simulates the signatures by programming the random oracle $\mathcal{H}$.

For a signature query on a message msg_j , the simulator $\mathcal{B}_2$ performs the following:

1. Uniformly sample a random vector $\mathbf{s}_j \xleftarrow{\$} \mathbb{F}_q^n$.
2. Evaluate the public map to compute the target vector $\mathbf{t}_j = \mathcal{P}^*(\mathbf{s}_j)$.
3. Program (back-patch) the random oracle such that $\mathcal{H}(\text{msg}_j \parallel \text{seed}_{pk}) := \mathbf{t}_j$. If the value $\mathcal{H}(\text{msg}_j \parallel \text{seed}_{pk})$ has already been defined by a previous query, the simulator aborts. However, since $\mathbf{t}_j \in \mathbb{F}_q^m$ is randomly distributed and $q_s \ll q^m$, the probability of such a collision is negligible.
4. Return $\sigma_j = (\mathbf{s}_j, \text{salt}_j)$ to $\mathcal{A}$ as the valid signature.

Because $\mathcal{P}^*$ maps a uniform distribution over $\mathbb{F}_q^n$ to an approximately uniform distribution over $\mathbb{F}_q^m$, the simulated signatures $(\mathbf{s}_j)$ are statistically indistinguishable from genuine signatures. Thus, Game 2 perfectly simulates Game 1 from $\mathcal{A}$'s perspective.

Eventually, $\mathcal{A}$ outputs a valid forgery $(\text{msg}^*, \mathbf{s}^*)$ such that the message was never queried to the signing oracle. For the signature to be valid, it must hold that $\mathcal{P}^*(\mathbf{s}^*) = \mathcal{H}(\text{msg}^* \parallel \text{seed}_{pk})$.

This implies that $\mathcal{A}$ has successfully found a preimage $\mathbf{s}^*$ for the target vector $\mathbf{t}^* = \mathcal{H}(\text{msg}^* \parallel \text{seed}_{pk})$ under the trapdoor-less map $\mathcal{P}^*$. The simulator $\mathcal{B}_2$ extracts $\mathbf{s}^*$ to trivially solve the given DOVE-MQ problem instance embedded in the public key. Therefore, $\mathcal{A}$'s advantage in Game 2 is bounded by:

$$\Pr[S_2] \leq \text{Adv}^{\text{DOVE-MQ}}(\mathcal{B}_2) + \frac{q_s \cdot q_h}{q^m}$$

By summing the advantages across these game transitions, we conclude that the overall EUF-CMA advantage of any PPT adversary $\mathcal{A}$ against the DOVE signature scheme is tightly bounded by:

$$\text{Adv}_{\text{DOVE}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \text{Adv}^{\text{DOVE-UOV}}(\mathcal{B}_1) + \text{Adv}^{\text{DOVE-MQ}}(\mathcal{B}_2) + \mathcal{O}(q^{-m})$$

This completes the proof. □

5 Implementations and Performance

5.1 Reference implementation

The performance evaluation of the DOVE reference implementation is conducted on a compliant environment as specified in the self-assessment guideline. The detailed software and hardware configurations are specified below to ensure the reproducibility of the benchmark results.

Software Configuration. The scheme is developed in ISO C99 and compiled using gcc (version 9.4.0) with the flags `-std=c99 -Wpedantic -Wall -Wextra -O2`. The operating system is Ubuntu 20.04.6 LTS running on Windows Subsystem for Linux (WSL2, kernel 6.18.33.2-microsoft-standard-WSL2). The build system uses CMake 3.16.3.

Cryptographic Primitives. For hash functions and pseudorandom number generation (PRNG), the underlying hash SM3 is utilized.

Hardware Platform. The benchmarks are executed on a 64-bit x86-64 platform (12th Gen Intel Core i9-12900H) with 15 GB RAM. Although the processor supports advanced vector instructions such as AVX2, this reference C implementation does not explicitly utilize SIMD-level optimizations.

Benchmarking Methodology. Each operation is run 100 times (≥ 100); signing uses a fixed 64-byte message. The reported CPU cycle counts represent the average values derived from 100 independent trials for each parameter set, ensuring a statistically stable performance profile.

Table 3 reports the average cycle counts and corresponding throughput (opt/s).

Table 3: Performance test results for reference implementation on DOVE parameter sets. Each operation is run 100 times; signing uses a fixed 64-byte message. Average CPU cycles (RDTSC) and throughput (ops/s) are reported.

Parameter set	Operation	Avg CPU cycles	ops/s
DOVE_classic_128	KeyGen	200,592,080	14.55
	Sign	13,111,398	222.58
	Verify	12,498,612	233.49
DOVE_classic_256	KeyGen	4,500,668,099	0.65
	Sign	146,716,668	19.89
	Verify	138,438,362	21.08
DOVE_classic_512	KeyGen	76,871,749,764	0.04
	Sign	1,087,512,275	2.68
	Verify	941,133,146	3.10
DOVE_pkc_skc_128	KeyGen	158,844,644	18.37
	Sign	46,245,244	63.11
	Verify	18,202,461	160.33
DOVE_pkc_skc_256	KeyGen	3,130,650,711	0.93
	Sign	670,849,202	4.35
	Verify	186,015,552	15.69
DOVE_pkc_skc_512	KeyGen	95,845,309,396	0.03
	Sign	12,236,132,118	0.24
	Verify	2,348,465,237	1.24

5.2 Performance-optimized implementation

The performance-optimized implementation leverages AVX2 vector instructions to accelerate the DOVE operations. The evaluation is conducted on the same hardware platform as the reference implementation, with the compiler configuration tuned for SIMD-level optimization.

Software Configuration. The scheme is developed in ISO C99 and compiled using gcc (version 9.4.0) with the flags `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`. The operating system is Ubuntu 20.04.6 LTS running on Windows Subsystem for Linux (WSL2, kernel 6.18.33.2-microsoft-standard-WSL2). The build system uses CMake 3.16.3.

Cryptographic Primitives. For hash functions and pseudorandom number generation (PRNG), the underlying hash SM3 is utilized.

Hardware Platform. The benchmarks are executed on a 64-bit x86-64 platform (12th Gen Intel Core i9-12900H) with 15.5 GB RAM. The processor supports advanced vector instructions including AVX2, which are explicitly utilized in this optimized implementation.

Benchmarking Methodology. Each operation is run 100 times (≥ 100); signing uses a fixed 64-byte message. The reported CPU cycle counts represent the average values derived from 100 independent trials for each parameter set, ensuring a statistically stable performance profile.

Table 4 reports the average cycle counts and corresponding throughput (ops/s). A more optimized constant-time implementation is left for future work.

Table 4: Performance test results for avx2 implementation on DOVE parameter sets. Each operation is run 100 times; signing uses a fixed 64-byte message. Average CPU cycles (RDTSC) and throughput (ops/s) are reported.

Parameter set	Operation	Avg CPU cycles	ops/s
DOVE_classic_128	KeyGen	42,329,477	68.94
	Sign	2,764,061	1,055.75
	Verify	5,781,061	504.79
DOVE_classic_256	KeyGen	457,651,534	6.38
	Sign	21,295,644	137.04
	Verify	28,217,290	103.42
DOVE_classic_512	KeyGen	6,760,373,809	0.43
	Sign	250,848,590	11.63
	Verify	421,307,321	6.93
DOVE_pkc_skc_128	KeyGen	28,994,237	100.65
	Sign	14,959,585	195.08
	Verify	13,260,626	220.07
DOVE_pkc_skc_256	KeyGen	281,021,579	10.38
	Sign	135,285,378	21.57
	Verify	112,308,877	25.99
DOVE_pkc_skc_512	KeyGen	4,858,325,659	0.60
	Sign	1,532,800,858	1.90
	Verify	1,358,369,362	2.15

6 Concrete Security Analysis

6.1 Classical Security Analysis

For a self-contained section, we first set the background. Given parameters (q, n, v, o, m) where $v = n - o$, let $\mathcal{P} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ be a homogeneous quadratic map that vanishes on an o -dimensional subspace $\mathcal{O}$, called the oil space. Define its *polar form* as $\mathcal{P}' : \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ such that

$$\mathcal{P}'(\mathbf{x}, \mathbf{y}) = \mathcal{P}(\mathbf{x} + \mathbf{y}) - \mathcal{P}(\mathbf{x}) - \mathcal{P}(\mathbf{y}) + \mathcal{P}(\mathbf{0}).$$

Furthermore, let $\mathcal{P} = (p_1, \dots, p_m)^\top$ be represented by matrices $\mathbf{P}_i \in \mathbb{F}_q^{n \times n}$ as $p_i(\mathbf{x}) = \mathbf{x}^\top \mathbf{P}_i \mathbf{x}$. Let $\mathcal{P}' = (p'_1, \dots, p'_m)^\top$, and let p'_i be represented by $\mathbf{M}_i \in \mathbb{F}_q^{n \times n}$ such that $p'_i(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{M}_i \mathbf{y}$. Then $\mathbf{M}_i = \mathbf{P}_i + \mathbf{P}_i^\top$, and p'_i is bilinear and symmetric; when $\text{char}(\mathbb{F}_q) = 2$, it is also alternating.

Generic attacks against UOV-like schemes are usually divided into direct attacks and key-recovery attacks. *Direct attacks* look for a solution to $\mathcal{P}(\mathbf{x}) = \mathbf{t}$ for a given target $\mathbf{t} \in \mathbb{F}_q^m$. In this note, $\text{MQ}(q; N, M)$ denotes the problem of solving M quadratic equations in N variables over $\mathbb{F}_q$. Thus a direct attack on the public map corresponds to an $\text{MQ}(q; n, m)$ instance. *Key-recovery attacks* aim to reconstruct the o -dimensional oil space $\mathcal{O} \subseteq \mathbb{F}_q^n$ such that $\mathcal{P}(\mathbf{o}) = \mathbf{0}$ for all $\mathbf{o} \in \mathcal{O}$.

And under the double vinegar-by-vinegar framework, we can take DOVE instances as UOV instances to analyze.

In this section, we comb through the various state-of-the-art algorithms and their complexity which is shown in Table 5 and Table 6.

Table 5: Bit-complexity estimates (lower bound for the base-2 logarithm of the number of binary classical gates required to perform an attack) of state-of-the-art attacks against our proposed parameter sets. Here the boldfaced numbers in each row represent the minimum bit-complexity estimates of various attacks against the related parameter set. And the ICCS security levels 1 (*resp.* 2 and 3) requires at least 2^{128} (*resp.* 2^{256} and 2^{512}) classical gates for a classical attacker to break the related scheme.

Attacks \ Parameter Sets (n, m, q)	DOVE ₁₂₈ (112, 44, 256)	DOVE ₂₅₆ (256, 96, 256)	DOVE ₅₁₂ (576, 216, 256)
Collision	186	395	876
Direct	145	277	577
Kipnis-Shamir	218	541	1184
Reconciliation	354	845	1815
Intersection	165	390	809
Wedge-Reconciliation	142	290	589
Wedge-Intersection	128	263	522

6.1.1 Direct attacks

6.1.1.1 Claw finding and collision attacks Considering DOVE signature schemes, one can also ask whether the hash function can be bypassed. One approach is a simple collision attack on the equality $P(s) = \text{Hash}(\mu \parallel \text{salt})$. Since that p is homogeneous, it suffices to find a collision up to a scalar multiple. This reduces the collision space to $q^m / (q - 1)$. An attacker can compute $P(s_i)$ for X inputs $\{s_i\}_{i \in [X]}$ and compute $\text{Hash}(\mu \parallel \text{salt}_j)$ for Y salts $\{\text{salt}_j\}_{j \in [Y]}$. If $X \cdot Y = \alpha q^m / (q - 1)$, then there is a collision $aP(s_i^*) = \text{Hash}(\mu \parallel \text{salt}_j^*)$ with probability $\approx 1 - e^{-\alpha}$ for some $a \in \mathbb{F}_q^\times$, and the attacker can output the signature $(\sqrt{a}s_i^*, \text{salt}_j^*)$ for the message μ .

Table 6: Bit-complexity estimates (lower bound for the base-2 logarithm of the number of binary quantum gates required to perform an attack) of state-of-the-art attacks against our proposed parameter sets. Here the boldfaced numbers in each row represent the minimum bit-complexity estimates of various attacks against the related parameter set. And the ICCS security levels 1 (*resp.* 2 and 3) requires at least 2^{80} (*resp.* 2^{128} and 2^{256}) quantum gates for a quantum attacker to break the related scheme.

Attacks \ Parameter Sets (n, m, q)	DOVE ₁₂₈ (112, 44, 256)	DOVE ₂₅₆ (256, 96, 256)	DOVE ₅₁₂ (576, 216, 256)
Direct	120	233	486
Kipnis-Shamir	122	285	608
Intersection	163	387	809
Wedge-Reconciliation	142	290	589
Wedge-Intersection	128	263	522

For the sake of concreteness, we say that the cost of a Keccak-f 1600 permutation is $2^{17.5}$ bit operations [11], so computing the list of hashes takes at least $Y \cdot 2^{17.5}$ bit operations.

Using Gray-code enumeration [12], we can evaluate a multivariate quadratic polynomial on a large number of inputs using only $3r$ bit operations per evaluation ($2r$ bit operations to compute the evaluation, and r bit operations to copy the evaluation to a list). We can optimize the attack by evaluating only the first $m' = m/2 + o(m)$ polynomials of P (as opposed to all m of them) to look for partial collisions (i.e., s_i and salt_j such that the first m' elements of $P(s_i)$ match the first $m' \log q$ bits of $\text{Hash}(\mu \parallel \text{salt}_j)$). Each time a partial collision is found, we use naive polynomial evaluation to check if it is a complete collision. For appropriately chosen m' , this second step is cheap because the number of partial collisions is small, therefore we ignore the second step in our cost analysis. The cost of the polynomial evaluations is $\frac{3mr}{2}X$ bit operations.

The lowest conceivable bit-cost of the total attack is then:

$$\frac{1}{(1 - e^{-\alpha})} \left(\frac{3mr}{2}X + 2^{17.5}Y \right),$$

which is approximately equal to $2^{10.7} \sqrt{q^m m r / (q - 1)}$ for optimally chosen X , Y , and α . This above formula should be interpreted as a conservative lower bound for the “true” cost of the attack.

6.1.1.2 Solve MQ problem Here we give a brief introduction on the state-of-the-art attacks on both overdetermined and underdetermined cases.

Firstly, when dealing with overdetermined cases where $m \geq n$, XL [13] and algorithms using Gröbner basis techniques such as F4 [14] and F5 [15] are of interest. In particular, among variants of XL algorithms, Wiedemann-XL [16] yields a time complexity

$$\min_{c \in [0, n]} \left\{ q^c \cdot 3 \binom{n - c + 2}{2} \binom{d_{reg} + n - c}{d_{reg}}^2 (2t^2 + t) \right\}$$

where $d_{reg} = d_{reg}(n - c, m)$ is the degree of regularity of related system estimated from the corresponding Hilbert series which is defined as the degree d of the first non-positive term z^d in the following series:

$$\frac{(1 - z^2)^m}{(1 - z)^{n - c + 1}}$$

and $t = \log_2 q$.

Next, for general underdetermined MQ problems, the Thomae-Wolf algorithm [17] is applicable. It transforms $\text{MQ}(q; n, m)$ with $n > m$ into an $\text{MQ}(q; m - \alpha, m - \alpha)$ instance, where $\alpha = \lfloor \frac{n}{m} \rfloor - 1$. The idea is to find a congruence transformation $\mathcal{S}$ such that $\mathcal{P} \circ \mathcal{S}$ consists of α matrices whose lower-right $m \times m$ submatrices have the form

$$\begin{bmatrix} \star & 0 & \cdots & 0 \\ 0 & \star & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \star \end{bmatrix}$$

and $m - \alpha$ remaining generic matrices. For the α matrices, the corresponding quadratic equations are like

$$x_{n-m+1}^2 + \cdots + x_n^2 + \sum_{j>n-m} L_j(x_1, \dots, x_{n-m})x_j + Q(x_1, \dots, x_{n-m}) = 0.$$

We find $x_1, \dots, x_{n-m}$ by solving the system of αm linear equations $L_j = 0$ in $n - m$ variables. So α is required to be no greater than $\frac{n}{m} - 1$. Then plugging the solution into the $m - \alpha$ remaining matrices, we obtain an $\text{MQ}(q; m - \alpha, m - \alpha)$ instance. Now we aim to find the matrix representation $\mathbf{S}$ of $\mathcal{S}$. Assume the left-most $n - m$ columns are $\begin{bmatrix} \mathbf{I}_{n-m} \\ \mathbf{0} \end{bmatrix}$ and the right-most m columns are $(\mathbf{s}_1, \dots, \mathbf{s}_m)$. Consider $\mathbf{S}^\top \mathbf{P} \mathbf{S}$, with its right-down submatrix as (6.1.1.2). So we have

$$\begin{bmatrix} \star & 0 & \cdots & 0 \\ 0 & \star & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \star \end{bmatrix} = \begin{bmatrix} \mathbf{s}_1^\top \\ \vdots \\ \mathbf{s}_m^\top \end{bmatrix} \mathbf{P}(\mathbf{s}_1, \dots, \mathbf{s}_m)$$

which implies that

$$\mathbf{s}_i^\top \mathbf{P} \mathbf{s}_j = 0 \quad \forall i \neq j.$$

Then we can sample $\mathbf{s}_m$ from $\mathbb{F}_q^n$ and find $\mathbf{s}_{m-1}, \dots, \mathbf{s}_1$ successively by solving the corresponding linear equations. Moreover, Hashimoto [18] proposed an improved approach for underdetermined MQ systems with parameters a and c satisfying

$$n \geq \max\{(a+1)(m-c-a+1), a(m-c) - (a-1)^2 + c\}$$

whose cost is expressed in terms of smaller MQ instances as

$$\min_{a,c} \{(m-a-c+1) \cdot \text{MQ}(q; a, a) + q^c \cdot (\text{MQ}(q; a-1, a-1) + \text{MQ}(q; m-a-c, m-a))\}.$$

And for the non-underdetermined MQ instance $\text{MQ}(q; a, a)$ and $\text{MQ}(q; m-a-c, m-a)$ instances, one can solve it using hybrid Wiedemann-XL algorithm with complexity mentioned above.

6.1.2 Key recovery attacks

We first analyze the UOV problem. Given parameters (q, n, o, m) and homogeneous quadratic polynomials $\mathcal{P} = (p_1, \dots, p_m) \in \mathbb{F}_q[x_1, \dots, x_n]$ that vanish on some o -dimensional subspace $\mathcal{O}$ of $\mathbb{F}_q^n$ of $\mathbb{F}_q^n$, the goal is to find such a subspace $\mathcal{O} \subseteq \mathbb{F}_q^n$ satisfying $\mathcal{P}(\mathcal{O}) = \mathbf{0}$, *i.e.*,

$$p_k(\mathbf{o}) = 0 \quad \text{for } k \in [m], \mathbf{o} \in \mathcal{O}.$$

Let $\mathbf{o}_1, \dots, \mathbf{o}_o$ be a basis of $\mathcal{O}$, and let $\mathcal{P}' = (q_1, \dots, q_m)$ be the polar form of $\mathcal{P}$, *i.e.*

$$q_k(\mathbf{x}, \mathbf{y}) = p_k(\mathbf{x} + \mathbf{y}) - p_k(\mathbf{x}) - p_k(\mathbf{y}).$$

Then we have

Proposition 1. Let $\mathbf{o}_1, \dots, \mathbf{o}_o$ be a basis of $\mathcal{O}$. Then $\mathcal{P}(\mathbf{o}) = \mathbf{0}$ for all $\mathbf{o} \in \mathcal{O}$ if and only if $\mathcal{P}(\mathbf{o}_i) = \mathbf{0}$ for all $i \in [o]$ and $\mathcal{P}'(\mathbf{o}_i, \mathbf{o}_j) = \mathbf{0}$ for all $i < j \in [o]$, i.e.,

$$\mathcal{P}(\mathbf{o}) = \mathbf{0} \quad \forall \mathbf{o} \in \mathcal{O} \Leftrightarrow \begin{cases} \mathcal{P}(\mathbf{o}_i) = \mathbf{0} & \text{for } i \in [o] \\ \mathcal{P}'(\mathbf{o}_i, \mathbf{o}_j) = \mathbf{0} & \text{for } i < j \in [o]. \end{cases}$$

In the proof, " $\Rightarrow$ " is trivial, and " $\Leftarrow$ " follows from

$$p_k \left(\sum_{i=1}^o a_i \mathbf{o}_i \right) = \sum_{i=1}^o a_i^2 p_k(\mathbf{o}_i) + \sum_{i>j} a_i a_j q_k(\mathbf{o}_i, \mathbf{o}_j) = 0.$$

Moreover, when the characteristic of $\mathbb{F}_q$ is odd, $\mathcal{P}'(\mathbf{o}_i, \mathbf{o}_i) = \mathbf{0}$ implies $\mathcal{P}(\mathbf{o}_i) = \mathbf{0}$ since $\mathcal{P}'(\mathbf{o}_i, \mathbf{o}_i) = 2\mathcal{P}(\mathbf{o}_i)$. In characteristic two, the polar form is alternating, so $\mathcal{P}'(\mathbf{o}, \mathbf{o}) = \mathbf{0}$ holds identically and the quadratic conditions $\mathcal{P}(\mathbf{o}) = \mathbf{0}$ must be kept separately.¹

Key recovery attacks aim to recover the oil space $\mathcal{O}$. P  bureau [19] proved that one single oil vector $\mathbf{o} \in \mathcal{O}$ is enough to recover an equivalent secret key, or equivalently the oil space, in polynomial time. Thus the central difficulty is often to find just one vector in $\mathcal{O}$.

6.1.2.1 Kipnis-Shamir attack Both the Kipnis-Shamir attack[4] and the intersection attack use the fact that $\mathbf{M}_i \mathcal{O} \subseteq \mathcal{O}^\perp$, where $\mathbf{M}_i$ is the matrix representation of p'_i .

There are two cases. When $o \geq v$, $\mathbf{M}_i \mathcal{O} = \mathcal{O}^\perp$, and $\mathbf{M}_j^{-1} \mathbf{M}_i \mathcal{O} = \mathcal{O}$ implies that $\mathcal{O}$ is an invariant subspace of the map $\mathbf{M}_j^{-1} \mathbf{M}_i$. This is the basic reason why balanced OV is insecure.

When $o < v$, the direct invariant-subspace argument no longer applies efficiently. Kipnis and Shamir claim that most vectors in the intersection $\mathbf{M}_i \mathcal{O} \cap \mathbf{M}_j \mathcal{O}$ are eigenvectors. As both $\mathbf{M}_i \mathcal{O}$ and $\mathbf{M}_j \mathcal{O}$ are subspaces of $\mathcal{O}^\perp$, the dimension of their intersection

$$\begin{aligned} \dim(\mathbf{M}_i \mathcal{O} \cap \mathbf{M}_j \mathcal{O}) &\leq \dim(\mathbf{M}_i \mathcal{O}) + \dim(\mathbf{M}_j \mathcal{O}) - \dim(\mathcal{O}^\perp) \\ &= 3m - n \end{aligned}$$

Then for an $\mathbf{x} \in \mathcal{O}$, the probability $\mathbf{M}_i \mathbf{x} \in \mathbf{M}_i \mathcal{O} \cap \mathbf{M}_j \mathcal{O}$ is roughly

$$\frac{|\mathbf{M}_i \mathcal{O} \cap \mathbf{M}_j \mathcal{O}|}{|\mathbf{M}_i \mathcal{O}|} = q^{-(n-2m)}$$

Hence the complexity of Kipnis-Shamir attack can be deduced as

$$q^{n-2o} n^\omega (2t^2 + t)$$

where $\omega = 2.8$ denotes the exponent of matrix multiplication and $t = \log_2(q)$.

6.1.2.2 Reconciliation attack The reconciliation attack [20] tries to solve the system of equations simultaneously

$$\begin{cases} \mathcal{P}(\mathbf{o}_i) = \mathbf{0} & \text{for } i \in [k] \\ \mathcal{P}'(\mathbf{o}_i, \mathbf{o}_j) = \mathbf{0} & \text{for } i, j \in [k] \text{ and } i < j \end{cases}.$$

Once an $\mathbf{o}$ satisfying the quadratic equations $\mathcal{P}(\mathbf{o}) = \mathbf{0}$ is found, it is much easier to solve for other $\mathbf{o}' \neq \mathbf{o}$ as linear relations are obtained through $\mathcal{P}'(\mathbf{o}, \mathbf{o}') = \mathbf{0}$. And without loss of generality, one can impose m linear affine constraints on each $\mathbf{o}_i$. For example, a reasonable approach is to assume $\mathbf{o}_1 = (o_{1,1}, \dots, o_{n-o,1}, 1, 0, \dots, 0)^\top$ and so forth.

Hence the complexity of reconciliation attack is

$$\min_k \text{MQ}(q, k(n-o), \binom{k+1}{2} m).$$

¹Let $\mathcal{P}'$ be the polar form of $\mathcal{P}$. If for all $\mathbf{o}, \mathbf{o}' \in \mathcal{O}$, $\mathcal{P}'(\mathbf{o}, \mathbf{o}') = \mathbf{0}$, we say $\mathcal{O}$ is a *totally isotropic subspace* for $\mathcal{P}'$. If for all $\mathbf{o} \in \mathcal{O}$, $\mathcal{P}(\mathbf{o}) = \mathbf{0}$, we say $\mathcal{O}$ is a *totally singular subspace* for $\mathcal{P}$.

6.1.2.3 Intersection attack Beullens' intersection attack [21] uses such intersections to construct systems of quadratic equations. Compared with direct inversion, the resulting MQ systems have more equations but fewer variables, and are then solved using XL-type algorithms.

There are also two cases. When $n < 3m$, intersected element $\mathbf{x} \in \mathbf{M}_i\mathcal{O} \cap \mathbf{M}_j\mathcal{O}$ is solution to

$$\begin{cases} \mathcal{P}(\mathbf{M}_i^{-1}\mathbf{x}) = \mathbf{0} \\ \mathcal{P}(\mathbf{M}_j^{-1}\mathbf{x}) = \mathbf{0} \\ \mathcal{P}'(\mathbf{M}_i^{-1}\mathbf{x}, \mathbf{M}_j^{-1}\mathbf{x}) = \mathbf{0} \end{cases}$$

and the problem turns into solving $3m$ quadratic equations in $2n - 3m$ variables, *i.e.*, an $\text{MQ}(q; 2n - 3o, 3m)$ instance.

When $n < 2.5m$, we consider element $\mathbf{x}$ in intersection of more than two sets. Specifically, let $\mathbf{L}_1, \dots, \mathbf{L}_k$ be linear combinations of $\mathbf{M}_i$'s and consider intersected element $\mathbf{x} \in \mathbf{L}_1\mathcal{O} \cap \dots \cap \mathbf{L}_k\mathcal{O}$ which satisfies

$$\begin{cases} \mathcal{P}(\mathbf{L}_i^{-1}\mathbf{x}) = \mathbf{0} \text{ for } i \in [k] \\ \mathcal{P}'(\mathbf{L}_i^{-1}\mathbf{x}, \mathbf{L}_j^{-1}\mathbf{x}) = \mathbf{0} \text{ for } i < j \end{cases}$$

$\mathbf{x} \in \mathbf{L}_1\mathcal{O} \cap \dots \cap \mathbf{L}_k\mathcal{O}$ will have dimension at least $km - (k-1)(n-m)$.

And if $\mathbf{L}_i = \sum_{t=1}^m c_{it}\mathbf{M}_t$ for each $i \in [1, m]$, then one can find two linear dependencies for each $i < j$ pair:

$$\sum_{t=1}^m c_{it}\mathcal{P}'_t(\mathbf{L}_i^{-1}\mathbf{x}, \mathbf{L}_j^{-1}\mathbf{x}) = \sum_{t=1}^m c_{jt}\mathcal{P}_t(\mathbf{L}_j^{-1}\mathbf{x})$$

Hence, the problem is transformed to solve a system of $\binom{k+1}{2}m - 2\binom{k}{2}$ quadratic equations in $kn - (2k-1)o$ variables, *i.e.*, an $\text{MQ}(q; kn - (2k-1)o, \binom{k+1}{2}m - 2\binom{k}{2})$ instance. We usually choose $k = 2$ or 3 for optimal estimation. What's more, following a similar approach in Kipnis-Shamir attack, we can repeatedly choose the target vector if no positive upper bound of $\dim(\cap_{i=1}^k \mathbf{L}_i\mathcal{O})$ is found. Hence the probabilistic approach leads to a complexity of

$$\begin{cases} \min_{k=2,3} q^{\max\{(k-1)(n-o)-ko+1,0\}} \cdot \text{MQ}(q; kn - (2k-1)o, \binom{k+1}{2}m - 2\binom{k}{2}) & \text{if } n < 3o \\ q^{n-3o+1} \cdot \text{MQ}(q; kn - (2k-1)o, \binom{k+1}{2}m - 2\binom{k}{2}) & \text{if } n \geq 3o \end{cases}$$

6.1.2.4 Rectangular MinRank attack The rectangular MinRank attack was introduced by Beullens [21] in the cryptanalysis of Rainbow and was later adapted to UOV variants, including MAYO and QR-UOV [22]. The attack reduces key recovery to a MinRank problem: given matrices $\mathbf{L}_1, \dots, \mathbf{L}_k \in \mathbb{F}_q^{n \times m}$ and a target rank $r < \min\{n, m\}$, find nonzero coefficients $y_1, \dots, y_k$ such that

$$\text{rank}(y_1\mathbf{L}_1 + \dots + y_k\mathbf{L}_k) \leq r.$$

A general technique used to solve MinRank problems is support-minors modeling. Suppose $\mathbf{y} = (y_1, \dots, y_k)^\top \in \mathbb{F}_q^k$ is the coefficient-vector we want. Let $\mathbf{L}_\mathbf{y} = \sum_{i=1}^k y_i\mathbf{L}_i = (\mathbf{L}_\mathbf{y}^{(1)}, \dots, \mathbf{L}_\mathbf{y}^{(n)})^\top$. Let $\mathbf{C} \in \mathbb{F}_q^{r \times m}$ be a basis matrix for the row space $\text{span}(\mathbf{L}_\mathbf{y}^{(1)}, \dots, \mathbf{L}_\mathbf{y}^{(n)})$. With *rank decomposition*, we have $\mathbf{S}\mathbf{C} = \mathbf{L}_\mathbf{y}$ where $\mathbf{S} \in \mathbb{F}_q^{n \times r}$ contains *support variables*, $\mathbf{C} \in \mathbb{F}_q^{r \times m}$ contains *minor variables*, and $y_1, \dots, y_k$ are *coefficient variables*.

Suzuki et al. [23] describe the rectangular MinRank attack in the same Jacobian framework as the Kipnis-Shamir and singular-point attacks. They extend the feasible target ranks

from the original formulation $\begin{cases} \mathcal{P}(\mathbf{x}) = \mathbf{0} \\ \text{rank}(\text{Rect}_{\text{col}}(\mathcal{P}^*\mathbf{x})) \leq v \end{cases}$ to $\begin{cases} \mathcal{P}(\mathbf{x}) = \mathbf{0} \\ \text{rank}(\text{Rect}_{\text{col}}(\mathcal{P}^*\mathbf{x})) \leq r \end{cases}$.

Combine their analysis with column truncating technique, the cost of support-minors approach is

$$\min_{\substack{r \leq n-o, \ D(r) > 0 \\ m' \in [r+1, m]}} 3 \binom{m'}{r}^2 \binom{n-D(r)+b_{\min}}{b_{\min}}^2 (r+1)(n-D(r)+1) \cdot (2t^2 + t),$$

where $t = \log_2(q)$, $D(r) = D_{n,m',o}(r) := o - (m' - r)(n - o - r)$ and $b_{\min}$ is the smallest integer satisfying $\mathcal{R}(b) > \mathcal{M}(b, 1)_{\text{col}}$. Here $\mathcal{R}(b)$ is the rank of bi-degree $(b, 1)$ Macaulay matrix of $\mathcal{M}(b, 1)$ which is approximated as

$$\mathcal{R}(b) := \sum_{i=1}^b (-1)^{i+1} \binom{m'}{r+i} \binom{n+i-1}{i} \binom{r+b-i-1}{b-i}.$$

$b_{\min}$ is well-estimated as the smallest integer b such that the coefficient of t^b in the power series

$$(1 - t^2)^{m'} \cdot \sum_{i \geq 0} (\mathcal{M}(i, 1)_{\text{col}} - \mathcal{R}(i)) t^i$$

is non-positive. And $\mathcal{M}(i, 1)_{\text{col}} = \binom{r}{m'} \binom{i}{n-D(r)+b}$ is the total number of monomials where the degree in x is i and the degree in y is 1. And here we note that MinRank attack on UOV instances is only effective when $v < m$.

6.1.3 Wedge, symmetric-algebra, and truncated-ring attacks

Ran's wedge attack [24] exploits the fact that in characteristic two the polar forms are alternating. Let $\alpha \in \wedge^v \mathbb{F}_q^n$ denote the exterior-algebra representative of the vinegar subspace. The attack considers the linear map

$$\mathcal{M} : \wedge^v \mathbb{F}_q^n \rightarrow (\wedge^{v+2} \mathbb{F}_q^n)^m, \quad \alpha \mapsto (\alpha \wedge q_1, \dots, \alpha \wedge q_m),$$

and recovers the secret subspace when the relevant kernel is small enough. This attack is specific to characteristic two and improves the security estimates for several UOV-like parameters.

For odd characteristic, Jin et al. [25] generalize the exterior-algebra viewpoint using the p -reduced symmetric algebra, which is isomorphic to

$$\mathbb{F}_q[x_1, \dots, x_n] / \langle x_1^p, \dots, x_n^p \rangle.$$

Their construction yields an XL-style key-recovery attack applicable to all characteristics.

Furue and Ikematsu [26] further generalize this framework to p^ℓ -truncated polynomial rings $\mathbb{F}_q[x_1, \dots, x_n] / \langle x_1^{p^\ell}, \dots, x_n^{p^\ell} \rangle$. The use of larger truncation levels can relax the solving-degree constraint and also gives a truncated-ring formulation of reconciliation attack as well as intersection attack. Take $t = \log_2(q)$ and $e = \log_p(q)$.

- First, for the reconciliation attack, the cost is estimated by

$$\min_{\substack{\ell \in [1, e], \\ k \in [0, o-1]}} 3 \binom{n-k+1}{2} \cdot (T(n-k, p^\ell, D) - T(o-k, p^\ell, D) + 1)^2 \cdot (2t^2 + t)$$

where o is the dimension of linear solution space,

$$T(n, p^\ell, d) = \sum_{i=0}^{\min\{n, \lfloor d/p^\ell \rfloor\}} (-1)^i \binom{n}{i} \binom{d - ip^\ell + n - 1}{n-1}$$

and

$$D = \min \left\{ d \in \mathbb{N} : \left\lfloor \frac{(1 + z + \dots + z^{p^\ell - 1})^{n-k}}{(1 + z^2 + \dots + z^{2(p^\ell - 1)})^m} \right\rfloor_d \leq T(o - k, p^\ell, d) \right\}.$$

- Next, for the intersection attack, the cost is estimated by

$$\begin{cases} \min_{\substack{\ell \in [1, e], \\ k \in [0, 2o - v - 1]}} 3 \binom{n-k+1}{2} \cdot (T(n - k, p^\ell, D) - T(2 * o - v - k, p^\ell, D) + 1)^2 \cdot (2t^2 + t) & \text{if } n < 3o \\ \min_{\ell \in [1, e]} 3q^{v-2o+1} \binom{n+1}{2} \cdot T(n, p^\ell, D)^2 \cdot (2t^2 + t) & \text{if } n \geq 3o \end{cases}$$

Here the solving degree D is chosen as the minimum integer $d \in [2, (2o - v - k)(p^\ell - 1)]$ satisfying

$$\left\lfloor \frac{(1 + z + \dots + z^{p^\ell - 1})^{n-k}}{(1 + z^2 + \dots + z^{2(p^\ell - 1)})^{3m-2}} \right\rfloor_d \leq T(2o - v - k, p^\ell, d)$$

in the $n < 3o$ case, and the minimum integer $d \in [2, p^\ell - 1]$ satisfying

$$\left\lfloor \frac{(1 + z + \dots + z^{p^\ell - 1})^n}{(1 + z^2 + \dots + z^{2(p^\ell - 1)})^{3m-2}} \right\rfloor_d \leq T(1, p^\ell, d) = 1$$

in the $n \geq 3o$ case.

6.2 Quantum Security Analysis

Known quantum attacks against UOV-like signature schemes mainly apply Grover's algorithm [27] to accelerate the exhaustive-search component of classical attacks, achieving a quadratic speedup that effectively halves the bit-security level. According to the Institute of Commercial Cryptography Standards (ICCS), a scheme is required to support three quantum-resistant security levels of at least 80, 128, and 256 bits. We satisfy these requirements by choosing parameters such that the classical security is at least 160, 256, and 512 bits, respectively, so that after applying Grover's speedup the concrete security remains above the required levels.

By applying Grover's algorithm to accelerate the mentioned approaches above, some cost estimates can be directly decreased to some extent:

- Direct attack (Hashimoto):

$$\min_{a, c} \left\{ q^{c/2} \cdot ((m - a - c + 1) \cdot \text{MQ}(q; a, a) + (\text{MQ}(q; a - 1, a - 1) + \text{MQ}(q; m - a - c, m - a))) \right\}.$$

- Direct attack (Wiedemann-XL):

$$\min_{c \in [0, n]} \left\{ q^{c/2} \cdot 3 \binom{n - c + 2}{2} \binom{d_{reg} + n - c}{d_{reg}}^2 (2r^2 + r) \right\}$$

- Kipnis-Shamir attack:

$$q^{(n-2o)/2} n^{2.8} (2r^2 + r)$$

- intersection attack:

$$\begin{cases} \min_{k=2,3} q^{\max\{(k-1)(n-o)-ko+1, 0\}/2} \cdot \text{MQ}(q; kn - (2k-1)o, \binom{k+1}{2}m - 2\binom{k}{2}) & \text{if } n < 3o \\ q^{(n-3o+1)/2} \cdot \text{MQ}(q; kn - (2k-1)o, \binom{k+1}{2}m - 2\binom{k}{2}) & \text{if } n \geq 3o \end{cases}$$

And the lowered cost estimate of direct attack eventually leads to an estimate drop in other attacks which reduces to solving an MQ instance such as intersection attack and reconciliation attack.

7 Summary and Future Work

In summary, the DOVE signature scheme presents a highly optimized, structurally advanced evolution of the traditional UOV framework. By introducing a structured matrix factorization technique based on upper-triangular base matrices, DOVE successfully inherits the core advantages of UOV—namely, exceptionally short signature sizes and a robust, well-understood multivariate trapdoor mechanism. Furthermore, it significantly elevates the scheme’s practical deployment viability by drastically reducing XOF expansion overhead, computational complexity, and active memory consumption during both signature generation and verification.

However, these substantial performance gains come with a theoretical trade-off that requires careful and ongoing scrutiny. The primary limitation of DOVE lies in the uncertainty surrounding the cryptographic resilience of its specific structural modifications. It remains an open question whether the explicit algebraic correlation introduced by the base matrix decomposition ($P_{i,1} \sim A_{k_1} A_{k_2}$) could be exploited by novel structural or algebraic attacks. While our current security reductions rely firmly on the DOVE-MQ and DOVE-UOV hardness assumptions, extensive third-party cryptanalysis—particularly regarding advanced intersection attacks or MinRank modeling tailored to this factorization—is necessary to ascertain whether this mathematical structure inadvertently introduces new vulnerabilities not present in standard, completely random UOV systems.

Additionally, while DOVE optimizes computation and memory, it still inherits the relatively large public key size characteristic of standard UOV. Recently, several advanced MPKC designs (such as the MAYO signature scheme) have successfully achieved highly compact public keys, though this reduction typically comes at a severe cost to signing and verification efficiency.

Looking forward, if DOVE’s matrix factorization structure withstands the test of time and rigorous cryptanalysis, it holds immense potential as a versatile optimization tool for these compact-key schemes. For instance, the small internal UOV maps utilized within MAYO could be redesigned using DOVE’s base matrix decomposition. This integration would significantly accelerate MAYO’s signing and verification processes, effectively compensating for its current computational disadvantages compared to standard UOV. Ultimately, combining DOVE’s high-speed algebraic structure with modern key-compression frameworks could pave the way for a new generation of multivariate schemes that simultaneously achieve minimal public key sizes, ultra-short signatures, and blazing-fast execution.

References

- [1] Jacques Patarin. The Oil and Vinegar signature scheme. In *Dagstuhl Workshop on Cryptography*, September 1997.
- [2] Jacques Patarin. Cryptanalysis of the Matsumoto and Imai public key scheme of Eurocrypt’88. In Don Coppersmith, editor, *Advances in Cryptology — CRYPTO ’95*, pages 248–261, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.
- [3] Tsutomu Matsumoto and Hideki Imai. Public quadratic polynomial-tuples for efficient signature-verification and message-encryption. In *Advances in Cryptology—EUROCRYPT’88: Workshop on the Theory and Application of Cryptographic Techniques Davos, Switzerland, May 25–27, 1988 Proceedings 7*, pages 419–453. Springer, 1988.
- [4] Aviad Kipnis and Adi Shamir. Cryptanalysis of the Oil and Vinegar signature scheme. In Hugo Krawczyk, editor, *Advances in Cryptology — CRYPTO ’98*, pages 257–266, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.
- [5] Jintai Ding, Jason E Gower, and Dieter S Schmidt. *Multivariate public key cryptosystems*, volume 25. Springer Science & Business Media, 2006.
- [6] Ward Beullens, Fabio Campos, Sofia Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. *Round 1 Additional Signatures, Post-Quantum Cryptography: Digital Signature Schemes*, 2023.
- [7] Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Tsuyoshi Takagi, Kan Yasuda, Toshiyuki Miyazawa, Tsunekazu Saito, and Akira Nagai. QR-UOV. *Round 1 Additional Signatures, Post-Quantum Cryptography: Digital Signature Schemes*, 2023.
- [8] Lih-Chung Wang, Chun-Yen Chou, Jintai Ding, Yen-Liang Kuan, Ming-Siou Li, Bo-Shu Tseng, Po-En Tseng, and Chia-Chun Wang. SNOVA - Proposal for NISTPQC: Digital Signature Schemes project. *Round 1 Additional Signatures, Post-Quantum Cryptography: Digital Signature Schemes*, 2023.
- [9] Jintai Ding, Ming-Shing Chen, Matthias Julias Kannwischer, Albrecht Petzoldt, Jacques Patarin, Dieter Schmidt, and Bo-Yin Yang. Rainbow. Technical report, National Institute of Standards and Technology, October 2020. Available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [10] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. UOV: Unbalanced Oil and Vinegar - Algorithm Specifications and Supporting Documentation Version 1.0. *Round 1 Additional Signatures, Post-Quantum Cryptography: Digital Signature Schemes*, 2023.
- [11] Nigel Smart. ‘bristol fashion’ mpc circuits.
- [12] Charles Bouillaguet, Hsieh-Chung Chen, Chen-Mou Cheng, Tung Chou, Ruben Niederhagen, Adi Shamir, and Bo-Yin Yang. Fast Exhaustive Search for Polynomial Systems in $\mathbb{F}_2$.
- [13] Nicolas Courtois, Alexander Klimov, Jacques Patarin, and Adi Shamir. Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 392–407. Springer, 2000.

- [14] Jean-Charles Faugère. A new efficient algorithm for computing gröbner bases (f4). *Journal of pure and applied algebra*, 139(1-3):61–88, 1999.
- [15] Jean-Charles Faugère. A new efficient algorithm for computing gröbner bases without reduction to zero (F5). In *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation*, pages 75–83. ACM, 2002.
- [16] Bo-Yin Yang, Owen Chia-Hsin Chen, Daniel J Bernstein, and Jiun-Ming Chen. Analysis of QUAD. In *International Workshop on Fast Software Encryption*, pages 290–308. Springer, 2007.
- [17] Enrico Thomae and Christopher Wolf. Solving underdetermined systems of multivariate quadratic equations revisited. In *International workshop on public key cryptography*, pages 156–171. Springer, 2012.
- [18] Yasufumi Hashimoto. An improvement of algorithms to solve under-defined systems of multivariate quadratic equations. Cryptology ePrint Archive, Paper 2021/1045, 2021.
- [19] Pierre Pébereau. One vector to rule them all: Key recovery from one vector in UOV schemes. Cryptology ePrint Archive, Paper 2023/1131, 2023.
- [20] Jintai Ding, Bo-Yin Yang, Chia-Hsin Owen Chen, Ming-Shing Chen, and Chen-Mou Cheng. New differential-algebraic attacks and reparametrization of Rainbow. In *International Conference on Applied Cryptography and Network Security*, pages 242–257. Springer, 2008.
- [21] Ward Beullens. Improved cryptanalysis of uov and rainbow. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 348–373. Springer, 2021.
- [22] Hiroki Furue and Yasuhiko Ikematsu. A new security analysis against MAYO and QR-UOV using rectangular minrank attack. In Junji Shikata and Hiroki Kuzuno, editors, *Advances in Information and Computer Security - 18th International Workshop on Security, IWSEC 2023, Yokohama, Japan, August 29-31, 2023, Proceedings*, volume 14128 of *Lecture Notes in Computer Science*, pages 101–116. Springer, 2023.
- [23] Toshihiro Suzuki, Hiroki Furue, Takuma Ito, Shuhei Nakamura, and Shigenori Uchiyama. An extended rectangular MinRank attack against UOV and its variants. In *International Workshop on Security*, pages 111–130. Springer, 2025.
- [24] Lars Ran. Wedges, oil, and vinegar – an analysis of UOV in the exterior algebra. Cryptology ePrint Archive, Paper 2025/1143, 2025.
- [25] Yi Jin, Yuansheng Pan, Xiaoou He, Boru Gong, and Jintai Ding. Security analysis on UOV families with odd characteristics: Using symmetric algebra. Cryptology ePrint Archive, Paper 2025/1137, 2025.
- [26] Hiroki Furue and Yasuhiko Ikematsu. Key recovery attacks on UOV using p^l -truncated polynomial rings. Cryptology ePrint Archive, Paper 2026/298, 2026.
- [27] Lov K Grover. Quantum mechanics helps in searching for a needle in a haystack. *Physical review letters*, 79(2):325, 1997.